PYTHON

1. Lists vs. Tuples (Mutability & Object Identity)

Key Concept: Lists [] are mutable (can be changed). Tuples () are immutable (cannot be changed after creation).
Memory References: Assigning L2 = L1 does not copy the list; both variables point to the *same object in memory* (id(L1) == id(L2)). Modifying L2 modifies L1. To make a true independent copy, use L1.copy().

Example Code:

```
# --- Lists & Reference Behavior ---
L1 = [1, 2, 3]
L2 = L1        # Both point to the same memory address
L2[0] = 99
print(L1)      # Output: [99, 2, 3] (L1 changed!)
```

```
# Making an independent copy
L3 = L1.copy()
L3[0] = 7
print(L1)      # Output: [99, 2, 3] (L1 is safe)
```

```
# --- Common List Operations ---
colors = ["red", "green"]
print(len(colors))      # Get length -> 2
colors.append("mercury")  # Adds to the end
print("red" in colors)    # Check if element exists -> True
```

```
# --- Tuples ---
T1 = (1, 2, 3, 2)
# T1[0] = 5      # ERROR! Tuples cannot be modified
print(T1.count(2)) # Count occurrences of a value -> 2
print(T1.index(3)) # Find index of a value -> 2
```

2. Dictionaries

Key Concept: Key-value pairs. Keys must be immutable (strings, numbers, tuples). You can dynamically populate dictionaries using loops or comprehensions.

Example Code:

```
# Generating a dictionary dynamically (i, i*i)
n = 4
my_dict = {}
for i in range(1, n + 1):
    my_dict[i] = i * i

print(my_dict)      # Output: {1: 1, 2: 4, 3: 9, 4: 16}
print(my_dict.keys()) # Output dict_keys([1, 2, 3, 4])
```

3. Loops and Control Flow (for...else, while, continue)

continue: Skips the rest of the current loop iteration and moves to the next one.
else block with loops: Executes only if the loop finishes naturally (without hitting a break statement). Excellent for finding prime numbers.

Example Code:

```
# --- Continue Statement ---
# Print numbers 100 to 105, skipping multiples of 5
for x in range(100, 106):
    if x % 5 == 0:
        continue
    print(x, end=" ") # Output: 101 102 103 104

print("\n--- Prime Finder (Loop Else) ---")
# Find prime numbers or print their first divisor
```

```
for num in range(2, 10):
    for i in range(2, num):
        if num % i == 0:
            print(f"{num} equals {i} * {num//i}")
            break # Hits break -> skips the 'else' block
    else:
        # Loop finished without breaking
        print(f"{num} is a PRIME number")
```

4. Classes and Object-Oriented Programming (OOP)

Key Concept: Building templates using class. Always use self as the first argument in methods to refer to the specific instance. Private/Internal variables often use prefixes like _ or _ _.

Example Code:

```
import math

class Circle:
    def __init__(self, radius):
        self.radius = radius # Instance variable
    def area(self):
        return math.pi * (self.radius ** 2)
    def perimeter(self):
        return 2 * math.pi * self.radius

# Using the class
c = Circle(5)
print(f"Area: {c.area():.2f}")      # Area: 78.54
print(f"Perimeter: {c.perimeter():.2f}") # Perimeter: 31.42
```

5. File Operations & Exception Handling (try...except)

Key Concept: Prevent programs from crashing when errors happen (e.g., file not found, division by zero).

Example Code:

```
# Writing to a file
with open("test.txt", "w") as f:
    f.write("Line 1: Hello World\nLine 2: Python Exam")
# Reading and error handling
try:
    with open("non_existent_file.txt", "r") as f:
        content = f.read()
except FileNotFoundError:
    print("Error captured: The file you want to open does not exist!")
except ZeroDivisionError:
    print("Error captured: Cannot divide by zero!")
```

6. Plotting with matplotlib & Data with numpy

Key Concept: Generating data ranges using numpy.arange(start, stop, step) and plotting them.

Example Code:

```
import numpy as np
import matplotlib.pyplot as plt

# Generate x domain from -5 to 5 with step 0.01
x = np.arange(-5, 5.01, 0.01)
y = x**2 + 2*x + 1

plt.plot(x, y, 'r-', label="y = x^2 + 2x + 1") # red solid line
plt.title("Polynomial Plot")
plt.xlabel("X Axis")
plt.ylabel("Y Axis")
plt.legend()
plt.grid(True)
# In written exams, remember the function sequence: plot -> labels -> show
# plt.show()
```

MATLAB

Paper Exam Tip: In MATLAB, **indexing starts at 1, not 0**! Also, a **semicolon** ; at the end of a line **suppresses output** in the command window.

1. Creating Matrices & Size Queries

size(A): Returns [rows, columns].
length(A): Returns the size of the *largest* dimension.
zeros(r, c): Creates an $r \times c$ matrix of zeros.

Example Code:

```
A = [1 2 3; 4 5 6]; % 2x3 matrix

sz = size(A);      % sz = [2, 3]
r_count = size(A,1); % 2 (rows)
c_count = size(A,2); % 3 (columns)
len = length(A);    % 3 (because columns 3 > rows 2)

V = 4:2:8;         % Vector starting at 4, step 2, ending at 8 -> [4, 6, 8]
```

2. Multi-Plotting & Subplots

hold on: Retains plots so you can overlay multiple lines on the same figure.
subplot(m, n, p): Divides the window into an $m \times n$ grid, and puts the current plot into position p.

Example Code:

```
x = -5:0.01:5;
y1 = x.^2 + 1; % Notice the dot '.' for element-wise power!
y2 = x.^3 + x.^2 + 1;

% Subplot configuration: 2 rows, 1 column
subplot(2, 1, 1);
plot(x, y1, 'r'); title('Plot 1');

subplot(2, 1, 2);
plot(x, y2, 'b'); title('Plot 2');
```

3. Advanced Indexing: Logical Indexing, find(), and sub2ind()

A(:): Flattens a matrix into a single column vector (down columns first).
Logical Indexing: A > 3 returns a matrix of 1s (true) and 0s (false).
find(): Returns the linear indices or row/column coordinates where a condition is met.
sub2ind(size, row, col): Converts subscripts (row, col) into a single linear index.

MATLAB counts linear indexes down columns first:

Given: $A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$	Index 1 = 1, Index 2 = 4 Index 3 = 2, Index 4 = 5 Index 5 = 3, Index 6 = 6

Example Code:

```
A = [1 2 3; 4 5 6];

idx = find(A == 4); % Returns linear index -> 2

% Finding row and col explicitly
[row, col] = find(A == 6); % row = 2, col = 3

% Converting row/col back to a linear index
linear_idx = sub2ind(size(A), 2, 3); % Returns 6

% Reset elements meeting a condition
A(A > 4) = 0; % A becomes [1 2 3; 4 0 0]
```

4. Signal Processing Basics (Images, Histograms & Aliasing)

imshow(I)/imagesc(I): Displays an image matrix. imagesc scales data to use the full colormap range.
eye(N): Generates an $N \times N$ identity matrix (1s on diagonal, 0s elsewhere).
histogram(I): Plots the count of each distinct value in a matrix.
unique(A): Returns a vector containing sorted unique values without repetitions.

Signal Generation & Sampling (Aliasing)

To generate a clean wave: ensure your sampling interval dt is significantly smaller than the wave's period $T = \frac{1}{f}$. If dt is too large, aliasing occurs, distorting the perceived frequency.

Example Code:

```
% --- Image Matrix Basics ---
I = eye(32); % 32x32 identity matrix
% imagesc(I); % Visualizes the white diagonal line

% --- Signal Generation ---
A = 325;      % Amplitude
f = 50;       % Frequency 50 Hz
% Period T = 1/50 = 0.02 seconds

% Case A: Perfect Sampling (dt is tiny compared to period)
dt1 = 0.0001;
t1 = 0:dt1:0.04;
y1 = A * sin(2 * pi * f * t1);

% Case B: Aliasing Happens (dt = 0.02 is exactly equal to the period!)
dt2 = 0.02;
t2 = 0:dt2:0.04;
y2 = A * sin(2 * pi * f * t2); % This will unhelpfully sample only zeros!
```